GyaanVault

SPPU M.Sc. Data Science — Free Study Material

Lab: Machine Learning

DS-555-MJP

Solved Practical Manual

Semester II · Lab (MJP)

Prepared by Siddheshwar Thaware · Free for SPPU students

This solved practical manual covers the Machine Learning lab assignments from the official SPPU M.Sc. Data Science syllabus. Each practical states the aim followed by a complete, runnable solution and the expected output. Use it as a reference — type the code yourself and experiment with the parameters to learn effectively.

Requirements: Python 3.x with numpy, pandas, matplotlib, scikit-learn, scipy installed (pip install numpy pandas matplotlib scikit-learn scipy).

Practicals 1–3: Scatter plot, null handling, categorical encoding

See DS-511 Practicals 1–3 for the same techniques (scatter plot, dropna, one-hot/label encoding) — reuse that code.

Practical 4: Simple Linear Regression — predict house price

```
import numpy as np
from sklearn.linear_model import LinearRegression
area  = np.array([500,750,1000,1250,1500]).reshape(-1,1)
price = np.array([25,35,45,52,62])   # in lakhs
m = LinearRegression().fit(area, price)
print("Price for 1100 sqft:", m.predict([[1100]])[0].round(2))
```

Practical 5: Multiple Linear Regression

```
from sklearn.datasets import fetch_california_housing
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
X,y = fetch_california_housing(return_X_y=True)
Xtr,Xte,ytr,yte = train_test_split(X,y,test_size=.2,random_state=0)
m = LinearRegression().fit(Xtr,ytr)
print("R2 score:", round(m.score(Xte,yte),3))
```

Practical 6: Polynomial Regression

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
X = np.linspace(0,5,20).reshape(-1,1); y = (X.ravel()**2) + np.random.rand(20)
model = make_pipeline(PolynomialFeatures(2), LinearRegression()).fit(X,y)
print("R2:", round(model.score(X,y),3))
```

Practical 7: Naïve Bayes · Practical 8: Decision Tree

Use the GaussianNB and DecisionTreeClassifier solutions from DS-511 Practicals 4 & 5.

Practical 9: Linear SVM

```
from sklearn.svm import SVC
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
X,y = load_iris(return_X_y=True)
Xtr,Xte,ytr,yte = train_test_split(X,y,test_size=.3,random_state=1)
print("Linear SVM accuracy:", SVC(kernel="linear").fit(Xtr,ytr).score(Xte,yte))
```

Practical 10: Neural network decision boundary (two-moons)

Same as DS-511 Practical 6 — MLPClassifier with hidden_layer_sizes=(10,).

Practical 11: PCA — dimensionality reduction

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_iris
X,y = load_iris(return_X_y=True)
X2 = PCA(n_components=2).fit_transform(X)
print("Reduced shape:", X2.shape)   # (150, 2)
```

Practicals 12–14: KNN, k-Means, Agglomerative clustering

Reuse DS-511 Practicals 7, 8 and 9 respectively.